

**Escuela Politécnica Nacional**

**Facultad de Ingeniería en Sistemas**

## **INFORME TÉCNICO: EJERCICIO 2**

**Asignatura: Compiladores y Lenguajes**

**Autores: Aidan Carrasco, Felipe Merino y Mathew Verdezoto**

### **1. Implementación y Código Fuente**

El código fuente se estructuró para identificar comandos literales específicos dentro de un flujo de texto, ejecutando rutinas de impresión simples al encontrarlos.

- `%{ #include <stdio.h> %}`: Directiva que incluye la librería estándar de entrada y salida de C, necesaria para habilitar la función `printf` en las acciones del autómata.
- `pare { printf("Comando pare recibido\n"); }`: Regla que define la expresión regular exacta "pare". Si el flujo coincide carácter por carácter, se ejecuta la acción en C.
- `inicio { printf("Comando inicio ejecutado\n"); }`: Regla idéntica a la anterior, estructurada para la palabra "inicio".
- `int main() { yylex(); return 0; }`: Función principal en C que invoca al motor generado por Lex (`yylex()`) para que comience a leer la entrada estándar.

**Tipos de entradas que recibe:** El programa acepta y procesa correctamente cadenas de texto que contengan exactamente las palabras "inicio" y "pare" escritas en minúsculas.

**Cadenas con las que el código fallaría (Errores Léxicos):** El código no está diseñado para ignorar signos de puntuación o variaciones de capitalización.

- Cadena `Inicio` o `PARE`: Al ser sensible a mayúsculas y minúsculas, el autómata no reconocerá estas cadenas como comandos válidos y aplicará la regla por defecto (imprimir la cadena tal cual).
- Cadena `pare.`: El punto pegado a la palabra romperá la tokenización esperada en un contexto real, aunque Lex imprimirá "Comando pare recibido" seguido de un "." suelto en consola.

Código fuente del archivo `dos.l`:

```
fmerino@fmerino: ~/COMPICD2026A/LAB-LEX-CarrascoA-M...
GNU nano 8.4 dos.l
/* Declaraciones */
%{
#include <stdio.h> // necesario para realizar "printf" de abajo
%}

/* Reglas */
%%
pare    printf("Comando pare recibido\n");
inicio  printf("Comando inicio ejecutado\n");
%%

/*Codigo (main) */
int main()
{
    yylex();
    return 0;
}
```

Archivo de entrada:

```
fmerino@fmerino: ~/COMPICD2026A/LAB-LEX-CarrascoA-M...
GNU nano 8.4 entrada.txt
hola
inicio
saltar
correr
pare
fin
```

## 2. Capturas de Ejecución

Se procesó un archivo de texto con múltiples sentencias que combinaban los comandos a evaluar junto con palabras aleatorias.

**Ejecución final: Prueba de Procesamiento y Redirección (Consola y Archivo salida.txt):**

```
fmerino@fmerino:~/COMPICD2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/U
NO$ lex dos.l
fmerino@fmerino:~/COMPICD2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/U
NO$ gcc lex.yy.c -lfl
fmerino@fmerino:~/COMPICD2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/U
NO$ ./a.out < entrada.txt
hola
Comando inicio ejecutado

saltar
correr
Comando pare recibido

fin
```

#### Descripción:

- **Comportamiento por defecto de Lex (Echo):** El analizador reconoció correctamente `inicio` y `pare`, pero las palabras como `hola`, `saltar` o `correr`, al no tener una regla definida, fueron impresas directamente en la salida estándar, demostrando que Lex no destruye la información no mapeada por defecto.
- **Procesamiento secuencial:** El autómata leyó y procesó el archivo de texto respetando estrictamente el orden y los saltos de línea originales del documento de entrada.

### 3. Conclusión

El uso de expresiones regulares literales es funcional para el mapeo estricto de palabras reservadas. No obstante, se evidencia que los autómatas deben diseñarse incluyendo reglas de exclusión para manejar el "ruido" o caracteres irrelevantes; de lo contrario, el analizador generará un eco en la salida con cada cadena que no posea una transición definida.